

Master Prompt Guide

Phase-by-Phase · Session-by-Session · Copy-Paste Ready 18 Sessions · 4 Phases · Full Stack Build

For	Claude Code terminal (claude command) — NOT claude.ai chat
Total Sessions	18 sessions across 4 phases
Time Estimate	Phase 1: Day 1–7 · Phase 2: Day 8–21 · Phase 3: Day 22–45 · Phase 4: Day 46–90
How to use	Copy the GREEN box. Paste into Claude Code. Wait. Check outputs. Move to next session.
If it breaks	Use the AMBER box — paste that message into same Claude Code session to fix.
Author	Nihal Mishra — Markeye

00 How to Use This Guide — Read First

This guide contains **18 Claude Code sessions**. Each session is one task. Complete them in order. Do NOT skip sessions or combine them — each builds on the last.

Before You Start — One-Time Setup

- ✓ Install Claude Code: `npm install -g @anthropic-ai/claude-code`
- ✓ Create project folder: `mkdir phantom-x && cd phantom-x`
- ✓ Run Claude Code: `claude` (in the phantom-x folder)
- ✓ Have ready: Supabase URL + anon key, Decodo proxy credentials, Claude API key, Upstash Redis URL
- ✓ Create `.env` file in `phantom-x/` before Session 1 — Claude Code will ask you to fill values

■ **GOLDEN RULE:** One session = one task. When Claude Code finishes and asks 'what next?', DO NOT continue in the same session. Open a fresh session and paste the next prompt. This prevents context drift and broken code.

Phase	Sessions	What Gets Built	Milestone
Phase 1 — Foundation	S01–S05	Project scaffold, DB schema, Docker, Playwright, Proxy	Day 7: Tool works internally
Phase 2 — Core Engine	S06–S10	Campaign engine, AI messages, ban detection, n8n, inbox	Day 21: First paying client
Phase 3 — Agency	S11–S15	Multi-tenant, white-label, CRM, dashboard, enrichment	Day 45: \$10K MRR push
Phase 4 — Polish	S16–S18	Self-serve, analytics, hardening	Day 90: \$20K MRR

PHASE 1 Foundation — Days 1–7

Build the skeleton. By end of Phase 1 you have a working FastAPI + Playwright + Supabase stack running in Docker.

SESSION 01

Project Scaffold + FastAPI Skeleton

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

You are helping me build a LinkedIn automation platform called Phantom-X.

Tech stack: Python 3.12, FastAPI, Supabase (supabase-py), Redis (upstash-redis), Playwright (playwright), Docker Compose, Caddy reverse proxy.

Task: Create the complete project scaffold with this exact structure:

```
phantom-x/
  app/
    main.py          (FastAPI app entry point)
    config.py        (Settings from env vars using pydantic-settings)
  api/
    __init__.py
    routes/
      __init__.py
      accounts.py    (empty router, just scaffold)
      campaigns.py   (empty router, just scaffold)
      leads.py       (empty router, just scaffold)
      scrape.py      (empty router, just scaffold)
      ai.py          (empty router, just scaffold)
      inbox.py       (empty router, just scaffold)
      analytics.py   (empty router, just scaffold)
      webhooks.py    (empty router, just scaffold)
  core/
    __init__.py
    auth.py          (JWT auth middleware stub)
    database.py      (Supabase client singleton)
    redis_client.py  (Upstash Redis client singleton)
  services/
    __init__.py
    (empty for now)
  workers/
    __init__.py
    (empty for now)
  requirements.txt   (all deps listed below)
```

```
.env.example      (all required env vars, no values)
docker-compose.yml (services: api, playwright-worker, n8n, redis, caddy)
Dockerfile        (for the api service)
Dockerfile.worker (for playwright-worker service)
Caddyfile         (routes: api.*, app.*, n8n.*)
.gitignore
```

Requirements.txt must include:

```
fastapi==0.115.0
uvicorn[standard]==0.30.0
supabase==2.9.0
upstash-redis==1.3.0
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
pydantic-settings==2.5.0
httpx==0.27.0
playwright==1.48.0
python-dotenv==1.0.0
anthropic==0.39.0
celery==5.4.0
redis==5.1.0
```

.env.example must have these keys (empty values):

```
SUPABASE_URL=
SUPABASE_ANON_KEY=
SUPABASE_SERVICE_KEY=
REDIS_URL=
ANTHROPIC_API_KEY=
DECODO_USERNAME=
DECODO_PASSWORD=
JWT_SECRET=
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=1440
SLACK_WEBHOOK_URL=
```

config.py must load all these from env using pydantic-settings BaseSettings.

main.py must include all routers and a /health endpoint returning {"status": "ok"}.

docker-compose.yml: api on port 8000, n8n on 5678, caddy on 80/443.

Dockerfile: FROM python:3.12-slim, installs requirements, runs uvicorn.

Generate all files completely. No placeholders. Working code only.

✓ EXPECTED OUTPUT — Session complete when you see:

- phantom-x/ folder created with all files

- `python -m uvicorn app.main:app --reload` runs without errors
- GET `http://localhost:8000/health` returns `{"status": "ok"}`
- GET `http://localhost:8000/docs` shows Swagger UI with all routes listed
- `.env.example` file present with all 11 env var keys
- `docker-compose.yml` present with `api`, `n8n`, `redis`, `caddy` services

■ IF IT BREAKS — Tell Claude Code exactly this:

- `ImportError` on any module: 'Fix the import error in [file]. The module structure is [paste structure above]'
- Port already in use: 'Change the `api` port in `docker-compose.yml` to 8001 and update `Caddyfile`'
- Pydantic validation error: 'Fix `config.py` — use `model_config = ConfigDict(env_file=".env")` in `BaseSettings`'
- `uvicorn` not found: 'Add `uvicorn[standard]` to `requirements.txt` and reinstall'

SESSION 02

Supabase Database Schema + Migration Files

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: I have a FastAPI project at `phantom-x/`.

Supabase is my database. I need all tables created.

Task: Generate complete SQL migration files for these tables.

Create a folder: `phantom-x/supabase/migrations/`

Create these files in order:

FILE: `001_workspaces.sql`

```
CREATE TABLE workspaces (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  owner_id uuid,
  branding jsonb DEFAULT '{}',
  plan text DEFAULT 'starter' CHECK (plan IN ('starter', 'pro', 'agency')),
  created_at timestampz DEFAULT now()
);

ALTER TABLE workspaces ENABLE ROW LEVEL SECURITY;
```

FILE: `002_users.sql`

```
CREATE TABLE users (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id uuid REFERENCES workspaces(id) ON DELETE CASCADE,
  email text UNIQUE NOT NULL,
  password_hash text NOT NULL,
  role text DEFAULT 'member' CHECK (role IN ('owner', 'admin', 'member', 'client_view')),
  created_at timestampz DEFAULT now()
```

```

);

ALTER TABLE users ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users see own workspace" ON users
  FOR ALL USING (workspace_id = (SELECT workspace_id FROM users WHERE id = auth.uid()));

FILE: 003_linkedin_accounts.sql
CREATE TABLE linkedin_accounts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id uuid REFERENCES workspaces(id) ON DELETE CASCADE NOT NULL,
  label text NOT NULL,
  li_at_encrypted text NOT NULL,
  status text DEFAULT 'warming_up' CHECK (status IN ('active','warming_up','restricted','expired'))
,
  warmup_day int DEFAULT 1,
  daily_limit int DEFAULT 50,
  actions_today int DEFAULT 0,
  proxy_country text DEFAULT 'US',
  safety_score int DEFAULT 100,
  last_health_check timestamptz,
  created_at timestamptz DEFAULT now()
);

ALTER TABLE linkedin_accounts ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Accounts by workspace" ON linkedin_accounts
  FOR ALL USING (workspace_id = current_setting('app.workspace_id')::uuid);

FILE: 004_campaigns.sql
CREATE TABLE campaigns (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id uuid REFERENCES workspaces(id) ON DELETE CASCADE NOT NULL,
  name text NOT NULL,
  status text DEFAULT 'draft' CHECK (status IN ('draft','active','paused','completed','error')),
  account_ids uuid[] DEFAULT '{}',
  sequence jsonb DEFAULT '[]',
  daily_limit int DEFAULT 50,
  icp_description text,
  stats jsonb DEFAULT '{"sent":0,"connected":0,"replied":0,"meetings":0}',
  created_at timestamptz DEFAULT now(),
  started_at timestamptz
);

ALTER TABLE campaigns ENABLE ROW LEVEL SECURITY;

FILE: 005_leads.sql
CREATE TABLE leads (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id uuid REFERENCES workspaces(id) ON DELETE CASCADE NOT NULL,

```

```

campaign_id uuid REFERENCES campaigns(id) ON DELETE SET NULL,
profile_url text NOT NULL,
full_name text,
headline text,
current_title text,
company text,
location text,
about text,
recent_posts jsonb DEFAULT '[]',
email text,
email_confidence text CHECK (email_confidence IN ('verified','likely','guessed',null)),
icp_score int DEFAULT 0,
pipeline_stage text DEFAULT 'queued' CHECK (pipeline_stage IN
    ('queued','sent','connected','replied','interested','booked','closed','not_interested')),
sent_at timestampz,
connected_at timestampz,
replied_at timestampz,
message_sent text,
notes text,
account_id uuid REFERENCES linkedin_accounts(id),
created_at timestampz DEFAULT now(),
UNIQUE(workspace_id, profile_url)
);
ALTER TABLE leads ENABLE ROW LEVEL SECURITY;
CREATE INDEX idx_leads_workspace ON leads(workspace_id);
CREATE INDEX idx_leads_campaign ON leads(campaign_id);
CREATE INDEX idx_leads_stage ON leads(pipeline_stage);

```

FILE: 006_messages.sql

```

CREATE TABLE messages (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    lead_id uuid REFERENCES leads(id) ON DELETE CASCADE NOT NULL,
    direction text CHECK (direction IN ('outbound','inbound')),
    content text NOT NULL,
    sent_via uuid REFERENCES linkedin_accounts(id),
    created_at timestampz DEFAULT now()
);
ALTER TABLE messages ENABLE ROW LEVEL SECURITY;

```

FILE: 007_jobs.sql

```

CREATE TABLE jobs (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id uuid REFERENCES workspaces(id),
    type text CHECK (type IN ('scrape','connect','message','enrich','health_check')),

```

```

    status text DEFAULT 'queued' CHECK (status IN ('queued','running','done','failed','retrying')),
    payload jsonb DEFAULT '{}',
    result jsonb DEFAULT '{}',
    retries int DEFAULT 0,
    created_at timestamptz DEFAULT now(),
    completed_at timestamptz
);
CREATE INDEX idx_jobs_status ON jobs(status);
CREATE INDEX idx_jobs_type ON jobs(type);

```

Also generate: phantom-x/app/core/database.py

Complete file with Supabase client using SUPABASE_URL and SUPABASE_SERVICE_KEY from config.

Include typed helper functions:

```

- get_leads(workspace_id, filters) -> list
- get_campaigns(workspace_id) -> list
- upsert_lead(lead_data) -> dict
- update_lead_stage(lead_id, stage) -> dict
- create_job(job_data) -> dict
- update_job_status(job_id, status, result) -> dict

```

Use supabase-py v2 syntax. All functions async.

✓ EXPECTED OUTPUT — Session complete when you see:

- supabase/migrations/ folder with 7 SQL files
- app/core/database.py with all helper functions
- Run migrations: paste SQL files in Supabase dashboard SQL editor — no errors
- All 7 tables visible in Supabase Table Editor
- RLS enabled on all tables (shown as shield icon in Supabase)

■ IF IT BREAKS — Tell Claude Code exactly this:

- RLS policy error: 'Remove the RLS policy from [file] — just keep ENABLE ROW LEVEL SECURITY for now'
- UUID extension missing: 'Add CREATE EXTENSION IF NOT EXISTS "pgcrypto"; at top of 001_workspaces.sql'
- Supabase client error in database.py: 'Fix using supabase-py v2 — client = create_client(url, key), NOT Client(url, key)'
- Foreign key violation order: 'Run migrations in numbered order 001 through 007 — workspaces must exist before users'

SESSION 03

Auth System — JWT Login + Middleware

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: I have phantom-x/ FastAPI project. Supabase has users and workspaces tables.

I need complete JWT authentication.

Task: Build complete auth system.

1. app/core/auth.py – complete implementation:

- hash_password(password: str) -> str (bcrypt)
- verify_password(plain: str, hashed: str) -> bool
- create_access_token(data: dict, expires_delta: timedelta) -> str (JWT, HS256)
- verify_token(token: str) -> dict (raise HTTPException 401 if invalid/expired)
- get_current_user(token: Annotated[str, Depends(oauth2_scheme)]) -> dict (async, queries Supabase users table, returns {id, email, workspace_id, role})
- require_role(roles: list[str]) – dependency factory for role checking

2. app/api/routes/auth.py – complete implementation:

POST /auth/register

body: {email, password, workspace_name}

creates workspace, then user with role=owner

returns: {access_token, token_type, user: {id, email, workspace_id, role}}

POST /auth/login

body: {email, password}

returns: {access_token, token_type, user: {id, email, workspace_id, role}}

GET /auth/me

requires: Bearer token

returns: current user object

3. Add /auth router to app/main.py

4. Write test_auth.py in phantom-x/ root:

Test register -> login -> /me flow using httpx

Print: "AUTH OK" if all pass, error details if not.

Use python-jose for JWT. Token expiry from config JWT_EXPIRE_MINUTES.

workspace_id stored in JWT claims.

All DB calls via app/core/database.py functions.

No placeholder code – full working implementation.

✓ EXPECTED OUTPUT — Session complete when you see:

- POST /auth/register with new email returns access_token
- POST /auth/login with same credentials returns access_token
- GET /auth/me with Bearer token returns user object with workspace_id
- GET /auth/me without token returns 401
- python test_auth.py prints AUTH OK

■ IF IT BREAKS — Tell Claude Code exactly this:

→ jose.exceptions.ExpiredSignatureError: 'In create_access_token, change expires_delta to timedelta(minutes=int(settings.JWT_EXPIRE_MINUTES))'

→ 422 Unprocessable Entity on login: 'Change login body to use OAuth2PasswordRequestForm or fix the Pydantic model field names'

→ Supabase insert error on register: 'Check users table has password_hash column not password — update the INSERT query'

→ CORS error: 'Add CORSMiddleware to main.py: app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])

SESSION 04

Playwright + Stealth + Proxy Engine

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: phantom-x/ project exists. Auth works. Now building the browser engine.
This is the core – Playwright runs LinkedIn sessions through residential proxies.

Task: Build complete Playwright service.

1. app/workers/browser_engine.py – complete implementation:

Class LinkedInBrowser:

```
async def __init__(self, account_id: str, proxy_country: str = "US"):
    - Fetch li_at_encrypted from Supabase linkedin_accounts table
    - Decrypt li_at (use Fernet with JWT_SECRET as key – derive 32-byte key with hashlib.sha256)
    - Start Playwright async, launch Chromium
    - Apply stealth: use playwright-stealth if available, else manually:
        page.add_init_script(STEALTH_SCRIPT) where STEALTH_SCRIPT is a variable containing:
            Object.defineProperty(navigator, 'webdriver', {get: () => undefined});
            window.chrome = {runtime: {}};
            Object.defineProperty(navigator, 'plugins', {get: () => [1,2,3]});
    - Set proxy from DECODED_USERNAME/PASSWORD env vars:
        proxy = {
            "server": f"http://gate.decodeo.com:10001",
            "username": f"{DECODED_USERNAME}-country-{proxy_country}",
            "password": DECODED_PASSWORD
        }
    - Inject li_at cookie:
        await context.add_cookies([
            {
                "name": "li_at", "value": li_at,
                "domain": ".linkedin.com", "path": "/"
            }
        ])
    - Set viewport: 1280x900, User-Agent: real Chrome UA string
    - self.page = await context.new_page()
    - self.account_id = account_id
```

```

async def check_session_valid(self) -> bool:
    - Navigate to https://www.linkedin.com/feed/
    - Wait 2 seconds
    - If redirected to /login or /checkpoint: return False
    - If page title contains "LinkedIn": return True
    - Return False

async def scrape_profile(self, profile_url: str) -> dict:
    - Navigate to profile_url
    - Random delay: await asyncio.sleep(random.uniform(2, 5))
    - Extract: full_name, headline, about, current_title, company, location
    - Extract last 3 posts (if visible): text of each post
    - Return dict with all fields (None if not found)
    - Selectors to try for name: h1.text-heading-xlarge, h1[class*="name"]
    - Selectors for headline: div.text-body-medium[class*="subtitle"]
    - Use page.query_selector_all for posts: div[class*="feed-shared-update"]

async def send_connection_request(self, profile_url: str, message: str) -> dict:
    - Navigate to profile_url
    - Random delay: await asyncio.sleep(random.uniform(3, 7))
    - Find Connect button: look for button with text "Connect"
    - If not found: check for "More" dropdown -> "Connect" inside
    - Click Connect
    - If modal appears with "Add a note": click it, type message (max 300 chars)
    - Click Send / Done
    - Random delay after: await asyncio.sleep(random.uniform(2, 4))
    - Return {"status": "sent", "profile_url": profile_url}
    - On any exception: return {"status": "error", "error": str(e)}

async def check_ban_signals(self) -> str:
    - Navigate to https://www.linkedin.com/feed/
    - Check response status, URL, page content
    - If status 401: return "session_expired"
    - If URL contains "checkpoint": return "restricted"
    - If page has captcha element: return "captcha"
    - Return "ok"

async def close(self):
    - Close browser and playwright

2. app/workers/job_runner.py:
async def run_job(job: dict):
    - Route to correct function based on job["type"]
    - Types: "health_check", "scrape", "connect"

```

- On any exception: update job status to "failed" in Supabase, log error
- Wrap each job in try/except

3. Install playwright browsers:

Add to Dockerfile.worker: `RUN playwright install chromium --with-deps`

No placeholder code. Full working implementation.

Handle `ImportError` for `playwright-stealth` gracefully (it may not be installed).

✓ EXPECTED OUTPUT — Session complete when you see:

- `browser_engine.py` imports without error
- `LinkedInBrowser()` instantiates without crashing
- `check_session_valid()` returns `True` with a real `li_at` cookie (test manually)
- `scrape_profile()` returns dict with at least `full_name` and `headline`
- No 'webdriver' detection: run check at intoli.com/blog/not-automated/ — should show PASS

■ IF IT BREAKS — Tell Claude Code exactly this:

→ Playwright not found: 'Run: `pip install playwright && playwright install chromium`'

→ Proxy auth error (407): 'Check `DECODO_USERNAME` format — it should be your username only, not `username:password`'

→ Cookie injection not working (still redirected to login): 'Change cookie domain from `.linkedin.com` to `linkedin.com` (no leading dot) and test again'

→ Selector not found for Connect button: 'Update `send_connection_request` to try these selectors in order: `[data-control-name="connect"]`, `button[aria-label*="Connect"]`'

→ Fernet key error: 'Replace Fernet with simple XOR cipher for now — we will upgrade encryption in Phase 3'

SESSION 05

LinkedIn Accounts API + Docker Compose Full Stack

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: `phantom-x/` has FastAPI, auth, Supabase schema, browser engine.

Now building the accounts management API and getting everything running in Docker.

Task Part 1: `app/api/routes/accounts.py` – complete implementation:

GET `/accounts`

- Requires auth (`get_current_user` dependency)
- Returns all `linkedin_accounts` for current `workspace_id`
- Include: `id`, `label`, `status`, `warmup_day`, `daily_limit`, `actions_today`, `safety_score`, `last_health_check`

POST `/accounts`

- Body: `{label: str, li_at: str, proxy_country: str = "US"}`
- Encrypt `li_at` using Fernet (key = `SHA256(JWT_SECRET)[:32]`)

- Insert into linkedin_accounts with workspace_id from JWT
- Return created account (without li_at_encrypted)

DELETE /accounts/{account_id}

- Verify account belongs to current workspace
- Delete from Supabase
- Return {"deleted": true}

GET /accounts/{account_id}/health

- Launch LinkedInBrowser for this account
- Call check_session_valid() and check_ban_signals()
- Update status and last_health_check in Supabase
- Return {"account_id": ..., "valid": bool, "signal": str, "status": str}

PATCH /accounts/{account_id}/limit

- Body: {daily_limit: int}
- Validate: 10 <= daily_limit <= 100
- Update in Supabase

Task Part 2: Update docker-compose.yml to be complete and working:

services:

```
api:
  build: .
  ports: ["8000:8000"]
  env_file: .env
  depends_on: [redis]
  volumes: [".:/app:/app/app"]
  command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

playwright-worker:

```
build:
  context: .
  dockerfile: Dockerfile.worker
env_file: .env
depends_on: [redis, api]
```

n8n:

```
image: n8nio/n8n:latest
ports: ["5678:5678"]
environment:
  - N8N_BASIC_AUTH_ACTIVE=true
  - N8N_BASIC_AUTH_USER=admin
  - N8N_BASIC_AUTH_PASSWORD=phantomx123
```

```
- WEBHOOK_URL=http://localhost:5678
volumes: ["n8n_data:/home/node/.n8n"]

redis:
  image: redis:7-alpine
  ports: ["6379:6379"]
```

```
caddy:
  image: caddy:2-alpine
  ports: ["80:80", "443:443"]
  volumes:
    - ./Caddyfile:/etc/caddy/Caddyfile
    - caddy_data:/data
```

```
volumes:
  n8n_data:
  caddy_data:
```

Task Part 3: Test script `docker_test.py`:

- Start API (assume it's running on :8000)
- Register user, login, add a test account with a fake `li_at`
- Verify account appears in GET `/accounts`
- Print "DOCKER STACK OK" or error details

Run the full stack: `docker compose up -d`
All services must start without errors.

✓ EXPECTED OUTPUT — Session complete when you see:

- `docker compose up -d` starts all 4 services (`api`, `playwright-worker`, `n8n`, `redis`, `caddy`)
- All containers show 'healthy' or 'running' in `docker compose ps`
- POST `/accounts` returns created account with `id`
- GET `/accounts` returns list with the account
- GET `/accounts/{id}/health` returns `{valid: bool, signal: str}`
- `n8n` accessible at `http://localhost:5678` (`admin/phantomx123`)
- `python docker_test.py` prints DOCKER STACK OK

■ IF IT BREAKS — Tell Claude Code exactly this:

- Container exits immediately: 'Show me the full logs: `docker compose logs api`'
- Redis connection refused: 'In `redis_client.py` change `host` to `redis` (Docker service name) not `localhost`'
- `n8n` can't connect: 'Check `n8n` port `5678` is not in use: `lsof -i :5678 && docker compose restart n8n`'
- Caddy TLS error on `localhost`: 'Add to `Caddyfile`: `{ auto_https off }` at the top for local dev'

→ Playwright worker crashes: 'Add to Dockerfile.worker: RUN apt-get install -y libglib2.0-0 libnss3 libnspr4 libdbus-1-3 libatk1.0-0 libatk-bridge2.0-0'

PHASE 2 Core Engine — Days 8–21

Build the campaign brain. By end of Phase 2 you have a fully working outreach tool running real LinkedIn campaigns with AI messages.

SESSION 06

Campaign API + Job Queue System

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: phantom-x/ has auth, accounts API, browser engine, Docker stack all working.

Now building campaign management and async job queue.

Task Part 1: app/api/routes/campaigns.py – complete:

GET /campaigns

Returns all campaigns for workspace, include stats, status, created_at

POST /campaigns

Body: {

name: str,

account_ids: list[str], # linkedin_account UUIDs

sequence: list[{step: int, type: str, delay_days: int, use_ai: bool, template: str}],

daily_limit: int = 50,

icp_description: str = None

}

Validates: account_ids belong to current workspace

Creates campaign with status=draft

Returns created campaign

POST /campaigns/{id}/start

Changes status from draft/paused -> active

Dispatches first batch of jobs to queue

Returns {started: true, jobs_queued: int}

POST /campaigns/{id}/pause

Changes status -> paused

Cancels pending jobs for this campaign in queue

Returns {paused: true}

GET /campaigns/{id}

Returns campaign detail + live stats from leads table (count per pipeline_stage)

DELETE /campaigns/{id}


```
Only if status is draft or completed
Returns {deleted: true}
```

Task Part 2: app/core/queue.py – Job queue using Redis:

Using upstash-redis (not BullMQ – we are in Python):

- Simple Redis list as queue (RPUSH to add, BLPOP to consume)
- Queue name: "phantom_x:jobs"
- Priority queue: "phantom_x:jobs:priority" (for health checks)

Functions:

```
async def enqueue_job(job_type: str, payload: dict, priority: bool = False) -> str:
```

- Creates job record in Supabase jobs table
- Pushes job_id to Redis queue
- Returns job_id

```
async def get_next_job() -> dict | None:
```

- BLPOP from priority queue first, then main queue (timeout=1)
- Fetches full job from Supabase by id
- Updates status to "running"
- Returns job dict

```
async def complete_job(job_id: str, result: dict):
```

- Updates job status to "done", sets result and completed_at

```
async def fail_job(job_id: str, error: str, retries: int):
```

- If retries < 3: re-enqueue with retries+1
- Else: update status to "failed"

Task Part 3: app/workers/campaign_executor.py:

```
async def execute_campaign_batch(campaign_id: str, batch_size: int = 10):
```

- Fetch campaign from Supabase
- Get leads with pipeline_stage = 'queued' for this campaign (limit=batch_size)
- Check daily_limit not exceeded for today
- For each lead: enqueue_job("connect", {lead_id, campaign_id, account_id})
- Account rotation: assign accounts round-robin from campaign.account_ids

```
async def worker_loop():
```

- Infinite loop: get_next_job() -> route to correct handler -> complete or fail
- This runs in playwright-worker container as main process

Update Dockerfile.worker CMD to: python -m app.workers.campaign_executor

✓ EXPECTED OUTPUT — Session complete when you see:

- POST /campaigns creates campaign and returns it with id
- POST /campaigns/{id}/start changes status to active and returns jobs_queued count
- Jobs appear in Redis queue: redis-cli LLEN phantom_x:jobs shows > 0
- Supabase jobs table shows rows with status='queued'
- worker_loop() starts without error (test: python -m app.workers.campaign_executor)

■ IF IT BREAKS — Tell Claude Code exactly this:

- Redis BLPOP timeout error: 'Change BLPOP timeout from 0 to 1 second: await redis.blpop(queue_name, timeout=1)'
- Campaign account_ids validation fails: 'In POST /campaigns, fetch accounts with workspace_id filter and compare UUIDs as strings'
- Worker loop ImportError: 'Add __main__ block: if __name__ == "__main__": asyncio.run(worker_loop())'
- Supabase RLS blocks worker: 'In queue.py use SUPABASE_SERVICE_KEY not SUPABASE_ANON_KEY for the Supabase client'

SESSION 07

AI Message Personalization via Claude API

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: Campaign system and job queue are working.

Now adding Claude AI to write personalized connection messages.

Task: app/services/ai_service.py — complete implementation:

1. async def generate_connection_message(profile_data: dict, campaign_brief: str = None) -> str:

profile_data contains: full_name, headline, current_title, company,
location, about, recent_posts (list of strings)

Build this system prompt:

"You are an expert B2B sales copywriter. Write a LinkedIn connection request message.

Rules: Under 280 characters. Sound like a real human, not a bot. No emojis.

No generic openers like 'I came across your profile'. Reference something SPECIFIC from their profile — a recent post, their exact role, a specific achievement.

No pitch in the first message. Just genuine interest + reason to connect.

If no specific info available, write something curiosity-based about their industry.

Return ONLY the message text. Nothing else."

User message:

"Write a connection message for: {full_name}, {current_title} at {company}.

Their headline: {headline}

About them: {about[:300] if about else 'Not available'}

Recent post: {recent_posts[0] if recent_posts else 'None visible'}

Campaign context: {campaign_brief or 'B2B outreach'}"

```
Call anthropic client: model="claude-haiku-4-5-20251001", max_tokens=150
Return the message text (strip whitespace)
```

Fallback if API fails or returns >300 chars:

```
Return: f"Hi {first_name}, your work in {industry} caught my attention –
        would love to connect and learn more about what you're building."
```

2. `async def score_lead_icp(profile_data: dict, icp_description: str) -> int:`

Prompt: "Score this LinkedIn profile against the ICP (Ideal Customer Profile).

Return ONLY a number from 0-100. 100 = perfect match. 0 = completely wrong.

ICP: {icp_description}

Profile: {full_name}, {current_title} at {company}. {headline}"

```
model="claude-haiku-4-5-20251001", max_tokens=10
```

```
Parse response as int, clamp to 0-100
```

```
Return score
```

3. `async def generate_followup_message(profile_data: dict, connection_message: str, step: int) -> str:`

Different prompt based on step:

step=2 (first DM after connect): warm, reference their work, soft CTA

step=3 (follow-up if no reply): shorter, different angle, final attempt

Same character limits and rules as `generate_connection_message`

```
Return message text
```

4. Update `app/workers/job_runner.py`:

In the "connect" job handler, before sending:

- Call `generate_connection_message(profile_data, icp_description)`
- Store generated message as `lead.message_sent`
- Pass message to `send_connection_request()`
- After sending: update `lead.pipeline_stage` to 'sent'

5. POST `/ai/preview-message` endpoint in `app/api/routes/ai.py`:

Body: {`profile_url`: str, `campaign_brief`: str}

Scrapes profile, generates message, returns it without sending

Used by dashboard to preview before campaign starts

Use `ANTHROPIC_API_KEY` from config. Handle API errors gracefully (use fallback).

✓ EXPECTED OUTPUT — Session complete when you see:

- `generate_connection_message()` returns string under 280 chars

- Message references something specific from the profile (not generic)
- `score_lead_icp()` returns integer 0-100
- `POST /ai/preview-message` returns a message for a real LinkedIn URL
- Worker 'connect' jobs now include AI-generated message in DB (check `leads.message_sent`)

■ IF IT BREAKS — Tell Claude Code exactly this:

→ `anthropic.APIError 401: 'Check ANTHROPIC_API_KEY in .env — copy from console.anthropic.com'`

→ Message over 280 chars: 'Add to system prompt: CRITICAL: Your response must be under 280 characters total. Count carefully.'

→ Claude returns explanation instead of just message: 'Add to user message: Return ONLY the message. No preamble. No explanation. Just the message text.'

→ `score_lead_icp` returns non-integer: 'Wrap in try/except: try: return int(response.strip()); except: return 50'

SESSION 08

LinkedIn Scraper + Lead Import

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: AI messages work. Now building the lead discovery system.

Task Part 1: `app/services/linkedin_scraper.py` – complete:

```
async def scrape_search_results(browser: LinkedInBrowser, search_url: str, max_leads: int = 200) -> list[dict]:
```

- Navigate to `search_url` (LinkedIn people search or Sales Nav)
- Scroll and paginate to collect profiles up to `max_leads`
- For each result: extract `profile_url`, `full_name`, `headline`, `company`, `location`
- Random delay between page loads: `uniform(3, 6)` seconds
- Stop if no "Next" pagination button found
- Return list of dicts

```
async def scrape_full_profile(browser: LinkedInBrowser, profile_url: str) -> dict:
```

- Full profile scrape: all fields in leads table
- Extract `recent_posts`: find last 3 posts user made, store text only
- Return complete dict

Try these selectors for search results:

- Result cards: `li[class*="reusable-search__result-container"]`
- Name inside card: `span[aria-hidden="true"]`
- Profile link: `a[class*="app-aware-link"]`
- Headline: `div[class*="entity-result__primary-subtitle"]`

Handle: sales navigator URLs differently (`search.linkedin.com/sales/*`)

On selector not found: log warning, skip that field (don't crash)

Task Part 2: app/api/routes/scrape.py – complete:

POST /scrape/search

Auth required

Body: {search_url: str, max_leads: int = 200, campaign_id: str = None, account_id: str}

- Validates account_id belongs to workspace
- Dispatches "scrape" job to queue
- Returns {job_id: str, status: "queued"}

POST /scrape/profile

Auth required

Body: {profile_url: str, account_id: str}

- Dispatches scrape job for single profile
- Returns {job_id: str}

GET /leads

Auth required

Query params: campaign_id, stage, search (name/company text search), limit=50, offset=0

Returns paginated leads list + total count

POST /leads/import

Auth required

Body: multipart CSV file upload

CSV columns expected: profile_url, full_name, company, headline (others optional)

Bulk upsert to leads table (ignore duplicates by workspace+profile_url)

Returns {imported: int, skipped: int, errors: int}

Task Part 3: Update job_runner.py for "scrape" job type:

- Launch LinkedInBrowser for given account_id
- Call scrape_search_results or scrape_full_profile
- Upsert each lead to Supabase (using workspace_id from campaign)
- If campaign_id provided: set leads.campaign_id
- Update job status on complete
- Close browser after

✓ EXPECTED OUTPUT — Session complete when you see:

- POST /scrape/search returns job_id immediately (non-blocking)
- After worker processes job: leads appear in Supabase leads table
- GET /leads returns the scraped leads with correct workspace filtering
- POST /leads/import with a test CSV returns {imported: N} correctly
- Scraper collects at least 10 profiles from a real LinkedIn search URL

■ IF IT BREAKS — Tell Claude Code exactly this:

→ Selectors not finding results: 'Print page.content() to console in scrape_search_results and show me the first 2000 characters of the HTML'

→ LinkedIn shows 'Please verify' page: 'The li_at cookie expired — add a fresh li_at and retry'

→ CSV import encoding error: 'Add encoding detection: open(file, encoding=chardet.detect(raw)["encoding"])

→ Scraper gets 0 results: 'Try navigating to the search URL manually in the non-headless browser: await browser.page.goto(url); await browser.page.screenshot(path="debug.png") — show me the screenshot'

SESSION 09

Ban Detection + n8n Webhooks + Slack Alerts

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: Scraping and campaigns working. Now adding safety layer and integrations.

Task Part 1: app/services/safety_monitor.py – complete:

```
async def check_account_safety(account_id: str) -> dict:
    - Launch browser for account
    - check_ban_signals() -> get signal
    - If signal != "ok":
        Update linkedin_accounts.status accordingly
        Pause all active campaigns using this account_id
        Fire Slack alert (see below)
    - Update last_health_check timestamp
    - Update safety_score:
        "ok" -> 100, slow degradation if recent issues
        "restricted" -> 0
        "captcha" -> 50
    - Return {account_id, signal, status, safety_score}

async def send_slack_alert(message: str, severity: str = "warning"):
    - POST to SLACK_WEBHOOK_URL from config (if not set, just log)
    - Payload: {"text": f"[PHANTOM-X] [{severity.upper()}] {message}"}
    - severity colors: warning=#F59E0B, critical=#F87171, info=#10B981

async def daily_limit_check(account_id: str) -> bool:
    - Check linkedin_accounts.actions_today < daily_limit
    - Return True if can continue, False if at limit

async def increment_action_count(account_id: str):
    - Supabase: actions_today += 1 for account_id

async def reset_daily_counts():
    - Called by cron: reset actions_today=0 for all accounts
    - Enqueue this as a scheduled job at midnight IST
```

Task Part 2: app/api/routes/webhooks.py – n8n integration:

POST /webhooks/campaign-trigger

No auth required (uses webhook_secret header instead)

Body: {campaign_id: str, webhook_secret: str}

Validates webhook_secret matches config.WEBHOOK_SECRET

Starts the campaign (same as POST /campaigns/{id}/start)

Returns {triggered: true, campaign_id: str}

POST /webhooks/register

Auth required

Body: {url: str, events: list[str]}

events: ["campaign.completed", "lead.connected", "lead.replied", "account.restricted"]

Store in Supabase webhooks table (create this table too)

Returns {id: str, registered: true}

async def fire_webhook(event: str, payload: dict, workspace_id: str):

- Fetch all registered webhooks for workspace_id that include this event
- POST to each URL: {event, timestamp, data: payload}
- Timeout 5 seconds, don't retry on failure (fire and forget)
- Log failures to console

Task Part 3: Integrate firing everywhere:

- After lead connects: fire_webhook("lead.connected", {lead_id, name, campaign_id})
- After lead replies: fire_webhook("lead.replied", {lead_id, message_preview})
- After account restricted: fire_webhook("account.restricted", {account_id, label})
- After campaign completes: fire_webhook("campaign.completed", {campaign_id, stats})

Also: Add Supabase table webhooks:

id uuid PK, workspace_id uuid, url text, events text[], created_at timestampz

✓ EXPECTED OUTPUT — Session complete when you see:

- POST /webhooks/campaign-trigger with correct secret starts a campaign
- Slack alert fires when check_account_safety() detects a problem (test with invalid li_at)
- actions_today increments in Supabase after each connection request
- Campaign auto-pauses when account hits daily_limit
- fire_webhook() POSTs to n8n webhook URL with correct payload (test with ngrok or n8n webhook node)

■ IF IT BREAKS — Tell Claude Code exactly this:

→ Slack 404: 'Check SLACK_WEBHOOK_URL starts with https://hooks.slack.com/services/ — create one at api.slack.com/apps'

→ Webhook fires but n8n doesn't receive: 'Make sure n8n is running and webhook node is activated (not just saved) in n8n UI'

→ daily_limit_check always returns True: 'Use SUPABASE_SERVICE_KEY client for this query — RLS may be blocking the count read'

→ Cron reset not running: 'For now, add a GET /admin/reset-daily-counts endpoint and call it manually each morning'

SESSION 10

Reply Inbox + Follow-Up Sequence Engine

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: Full campaign pipeline works. Adding reply detection and follow-up sequences.

Task Part 1: app/services/inbox_monitor.py:

```
async def poll_linkedin_inbox(account_id: str, browser: LinkedInBrowser) -> list[dict]:  
    - Navigate to https://www.linkedin.com/messaging/  
    - Scrape conversation list: find conversations with unread indicator  
    - For each unread conversation:  
        Extract: sender_name, sender_profile_url, message_preview, thread_url  
    - Cross-reference with leads table: find lead by profile_url match  
    - If lead found and direction is inbound:  
        Insert into messages table (direction="inbound")  
        Update lead.pipeline_stage to "replied" and lead.replied_at  
        fire_webhook("lead.replied", {lead_id, message_preview, account_id})  
    - Return list of {lead_id, sender_name, message, matched: bool}
```

Selectors to try for messaging:

- Conversation list: ul[class*="msg-conversations-container"]
- Unread: li[class*="is-unread"]
- Sender name: span[class*="msg-conversation-listitem__participant-names"]

async def schedule_inbox_polling():

- Poll every 30 minutes for all active accounts
- Run as background task using asyncio

Task Part 2: app/services/sequence_engine.py:

```
async def process_sequence_step(lead_id: str, campaign_id: str):  
    - Fetch lead and campaign from Supabase  
    - Find next sequence step based on lead's current pipeline_stage  
    - Check if delay has passed (lead.connected_at + step.delay_days <= now)  
    - If ready:  
        Generate message via ai_service (or use template)  
        Send DM via browser.send_direct_message(profile_url, message)  
        Record in messages table (direction="outbound")
```


Log outbound message

- If lead has replied: skip (stop sequence)
- If lead marked not_interested: skip

Add to LinkedInBrowser in browser_engine.py:

async def send_direct_message(self, profile_url: str, message: str) -> dict:

- Navigate to profile_url
- Find "Message" button (look for button with aria-label containing "Message")
- Click it, wait for messaging modal
- Type message in textarea
- Click Send
- Return {"status": "sent"} or {"status": "error", "error": str(e)}

Task Part 3: app/api/routes/inbox.py – complete:

GET /inbox

Auth required

Query: limit=50, offset=0, unread_only=false

Returns messages with lead info (name, company, stage) joined

Sorted by created_at DESC

POST /inbox/{lead_id}/reply

Auth required

Body: {message: str, account_id: str}

Sends DM via browser for the account that connected with this lead

Records in messages table (direction="outbound")

Returns {sent: true}

PATCH /inbox/{lead_id}/stage

Auth required

Body: {stage: str} # one of the pipeline stages

Updates lead.pipeline_stage

Returns updated lead

✓ EXPECTED OUTPUT — Session complete when you see:

- poll_linkedin_inbox() returns list of conversations (even empty list is OK)
- Inbound messages appear in Supabase messages table after polling
- Lead pipeline_stage updates to 'replied' automatically when reply detected
- GET /inbox returns messages with lead details
- POST /inbox/{lead_id}/reply sends DM and records it in messages table
- Sequence engine skips leads that have already replied

■ IF IT BREAKS — Tell Claude Code exactly this:

→ Messaging selectors not found: 'LinkedIn updates their classes frequently. Try: await page.goto("https://www.linkedin.com/messaging/"); await page.screenshot(path="inbox_debug.png") — then inspect the screenshot and tell me the visible class names'

→ Reply sends to wrong account: 'Store account_id in messages table when outbound — then use that account for replies, not any account'

→ Sequence engine runs step too early: 'Double check timedelta: (datetime.now() - lead.connected_at).days >= step.delay_days — use timezone-aware datetimes (datetime.now(UTC))'

→ Background polling crashes the API: 'Move schedule_inbox_polling to the worker container, not the API — add to worker_loop() as a separate asyncio.Task'

PHASE 3 Agency Platform — Days 22–45

Add multi-tenant workspaces, white-label, CRM, analytics, email enrichment. This is when it becomes sellable to agencies.

SESSION 11

Multi-Tenant Workspaces + Role-Based Access

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: Full working tool. Now making it agency-ready with real multi-tenancy.

Task: Upgrade the auth and workspace system to support agencies managing multiple clients.

1. New endpoints in `app/api/routes/workspaces.py`:

POST `/workspaces`

Auth required, role=owner only

Body: `{name: str, client_email: str = None}`

Creates new workspace (sub-workspace under agency)

Optionally invites `client_email` as `client_view` role

Returns new workspace

GET `/workspaces`

Auth required

If `user.role = owner`: return all workspaces they own

If `user.role = member/client_view`: return just their workspace

POST `/workspaces/switch/{workspace_id}`

Auth required

Switch active workspace in JWT context

Returns new token with `workspace_id` updated

2. Update `auth.py`:

- JWT now includes `{user_id, workspace_id, role}`

- `get_current_user` returns all three

- Add `agency_id` to `workspaces` table: `ALTER TABLE workspaces ADD COLUMN agency_id uuid REFERENCES workspaces(id);`

- Agency workspace can have many client workspaces (`agency_id` set on client workspaces)

3. Update ALL Supabase queries in `database.py`:

- Every query MUST filter by `workspace_id` from JWT

- Create helper: `get_workspace_context(user: dict) -> str` (returns `workspace_id`)

- Never trust `workspace_id` from request body — always use JWT

4. `app/api/routes/users.py`:

`POST /users/invite`

Auth required, role=owner or admin

Body: {email: str, role: str}

Creates user account with temp password

Sends welcome email (use httpx to POST to Resend API if RESEND_API_KEY set, else log)

`GET /users`

Auth required, role=owner or admin

Returns all users in current workspace

`DELETE /users/{user_id}`

Auth required, role=owner

Removes user from workspace (soft delete: set active=false)

5. Add to `.env.example`:

`RESEND_API_KEY=`

`APP_URL=http://localhost:3000`

✓ EXPECTED OUTPUT — Session complete when you see:

- `POST /workspaces` creates sub-workspace linked to agency
- `GET /workspaces` returns only workspaces belonging to current user
- `POST /workspaces/switch/{id}` returns new JWT with different `workspace_id`
- All leads/campaigns queries return ONLY data from active workspace (verified by creating data in two workspaces and checking isolation)
- `POST /users/invite` creates user in correct workspace

■ IF IT BREAKS — Tell Claude Code exactly this:

→ Cross-workspace data leak: 'In every Supabase query, add `.eq("workspace_id", workspace_id)` — check every function in `database.py`'

→ `workspace/switch` returns old workspace data: 'create_access_token must include new workspace_id: data={"sub": user_id, "workspace_id": new_workspace_id, "role": role}'

→ Invite email not sending: 'Skip Resend for now — just print the invite URL to console: `{APP_URL}/accept-invite?token={token}`'

SESSION 12

Email Enrichment Pipeline

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: Multi-tenant working. Now adding email enrichment to leads.

Task: `app/services/enrichment.py` — complete waterfall enrichment:

```

async def enrich_lead_email(lead: dict) -> dict:
    Returns: {email: str | None, confidence: str, source: str}

    Try in order (waterfall – stop at first success):

    Step 1: Apollo.io free API
    GET https://api.apollo.io/v1/people/match
    Headers: {"X-Api-Key": config.APOLLO_API_KEY}
    Body: {"first_name": ..., "last_name": ..., "organization_name": lead.company,
           "domain": extract_domain(lead.company)}
    If email returned and not None: return {email, confidence="verified", source="apollo"}

    Step 2: Hunter.io
    GET https://api.hunter.io/v2/email-finder
    Params: {domain: extract_domain(lead.company), first_name: ..., last_name: ..., api_key: config
    .HUNTER_API_KEY}
    If confidence >= 0.7: return {email, confidence="likely", source="hunter"}

    Step 3: Pattern guess (last resort)
    Common patterns: first@domain, first.last@domain, flast@domain
    For each pattern: validate format only (no SMTP verify to avoid blocks)
    Return {email: best_guess, confidence="guessed", source="pattern"}

    If all fail: return {email: None, confidence: None, source: None}

    Helper: extract_domain(company_name: str) -> str
    Try to find company website from LinkedIn profile data if available
    Fallback: company_name.lower().replace(" ", "") + ".com"

    async def enrich_campaign_leads(campaign_id: str, workspace_id: str):
        Fetch all leads for campaign where email is null
        For each lead in batches of 10: enrich_lead_email()
        Update lead.email and lead.email_confidence in Supabase
        Sleep 0.5s between requests to respect rate limits
        Return {enriched: int, failed: int, total: int}

    New endpoints in app/api/routes/leads.py:

    POST /leads/{lead_id}/enrich
    Auth required
    Trigger enrichment for single lead
    Return {email, confidence, source} or {email: null, message: "not found"}

    POST /campaigns/{campaign_id}/enrich-all
    Auth required

```

```
Dispatches background job: enrich_campaign_leads(campaign_id)

Return {job_id: str, leads_to_enrich: int}
```

Add to .env.example:

```
APOLLO_API_KEY=
HUNTER_API_KEY=
```

Handle: missing API keys gracefully (skip that step in waterfall).

Handle: API rate limit (429) — wait 60 seconds and retry once.

✓ EXPECTED OUTPUT — Session complete when you see:

- enrich_lead_email() returns dict with email field (even if None)
- Apollo API step runs without error when APOLLO_API_KEY is set
- POST /leads/{id}/enrich updates the lead in Supabase
- POST /campaigns/{id}/enrich-all dispatches job and returns job_id
- Supabase leads table shows email and email_confidence after enrichment job runs

■ IF IT BREAKS — Tell Claude Code exactly this:

→ Apollo 401: 'Apollo free tier needs X-API-Key header not Authorization — double check header name'

→ Hunter returns 422: 'Domain extraction failing — hardcode domain for test: pass domain="anthropic.com" to Hunter'

→ Pattern guess creates invalid emails: 'Add validation: import re; re.match(r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\$", email)'

→ Batch enrichment too slow: 'Reduce batch to 5 leads at a time and increase sleep to 1s — Apollo free tier is strict'

SESSION 13

Analytics API + Campaign Dashboard Data

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: All core features working. Building analytics layer so dashboard has real data.

Task: app/api/routes/analytics.py — complete:

GET /analytics/campaign/{campaign_id}

Auth required

Returns:

```
{
  campaign_id: str,
  name: str,
  status: str,
  summary: {
    total_leads: int,
    sent: int,
    connected: int,
```

```

    replied: int,
    meetings_booked: int,
    accept_rate: float, # connected/sent as percentage
    reply_rate: float, # replied/connected as percentage
},
by_stage: {stage: count for each stage},
daily_stats: [ # last 14 days
    {date: str, sent: int, connected: int, replied: int}
],
top_companies: [ # top 5 companies by connections
    {company: str, count: int}
],
account_breakdown: [ # per account in campaign
    {account_id: str, label: str, sent: int, connected: int, accept_rate: float}
]
}
Compute all from leads table (no cached stats – compute live)

```

GET /analytics/workspace

Auth required

Returns workspace-level aggregate:

```

{
    active_campaigns: int,
    total_leads_this_month: int,
    total_connected_this_month: int,
    total_replied_this_month: int,
    avg_accept_rate: float,
    avg_reply_rate: float,
    accounts: [
        {account_id, label, status, safety_score, actions_today, daily_limit}
    ]
}
"this_month" = current calendar month

```

GET /analytics/safety

Auth required

Returns all accounts with safety metrics:

```

{accounts: [{id, label, status, safety_score, actions_today, daily_limit, last_health_check}]}

```

Also: Create a simple analytics helper in database.py:

```

async def get_campaign_lead_counts(campaign_id: str) -> dict:

```

Single SQL query using COUNT + CASE WHEN to get all stage counts in one trip:

```

SELECT
    COUNT(*) as total,

```

```

COUNT(*) FILTER (WHERE pipeline_stage='sent') as sent,
COUNT(*) FILTER (WHERE pipeline_stage='connected') as connected,
COUNT(*) FILTER (WHERE pipeline_stage='replied') as replied,
COUNT(*) FILTER (WHERE pipeline_stage='booked') as meetings
FROM leads WHERE campaign_id = $1

```

Use Supabase RPC or raw SQL via postgres for complex queries.
 All rate math: handle division by zero (return 0.0 if denominator is 0).
 Return all percentages as 0-100 (not 0.0-1.0).

✓ EXPECTED OUTPUT — Session complete when you see:

- GET /analytics/campaign/{id} returns complete stats object
- accept_rate and reply_rate are correct percentages (test: 5 connected / 10 sent = 50%)
- daily_stats returns 14 rows (one per day, zeros if no activity)
- GET /analytics/workspace returns aggregate across all campaigns
- No division by zero errors when campaigns have 0 sends

■ IF IT BREAKS — Tell Claude Code exactly this:

→ Supabase COUNT FILTER syntax error: 'Use Python to compute: fetch all leads, then count in-memory with collections.Counter(lead["pipeline_stage"] for lead in leads)'

→ daily_stats has gaps (missing days): 'Generate all 14 dates in Python first, then left-join with DB results: {date: counts.get(date, {"sent":0,"connected":0,"replied":0})}'

→ accept_rate shows as integer not float: 'Divide as float: round(connected / max(sent, 1) * 100, 1)'

→ Analytics endpoint slow: 'Add index if not exists: CREATE INDEX idx_leads_campaign_stage ON leads(campaign_id, pipeline_stage)'

SESSION 14

White-Label + Branding System

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: Multi-tenant, analytics, enrichment all working.

Adding white-label so agency clients see agency brand, not Markeye.

Task Part 1: Update workspaces to support white-label config:

```

ALTER TABLE workspaces ADD COLUMN IF NOT EXISTS branding jsonb DEFAULT '{}';
branding schema: {
  logo_url: str,
  primary_color: str, # hex like #7C3AED
  product_name: str, # displayed in dashboard header
  support_email: str,
  custom_domain: str # e.g. "outreach.agencyname.com"
}

```


New endpoints in app/api/routes/workspaces.py:

GET /workspace/branding

Auth required

Returns current workspace branding config

PATCH /workspace/branding

Auth required, role=owner or admin

Body: partial branding object

Updates branding in Supabase (merge, not replace)

Returns updated branding

GET /workspace/branding/public

NO auth required (used by dashboard to load branding before login)

Requires: ?domain=outreach.agencyname.com OR ?workspace_id=xxx

Returns: {product_name, logo_url, primary_color} (no sensitive data)

Task Part 2: app/services/email_service.py – white-label email:

```
async def send_alert_email(to: str, subject: str, body: str, workspace_id: str):
```

- Fetch workspace branding

- If RESEND_API_KEY set: send via Resend API

 - From: f"{branding.product_name} <alerts@{branding.support_email or 'markeye.space'}>"

- Else: log to console with full email content

- HTML template: simple dark theme with workspace logo + colors

Task Part 3: Caddyfile update for custom domains:

Add wildcard handling comment explaining how to add custom domain:

```
# To add client domain: outreach.clientagency.com
```

```
# 1. Client sets DNS CNAME: outreach.clientagency.com -> app.markeye.space
```

```
# 2. Add to Caddyfile:
```

```
# outreach.clientagency.com {
```

```
#   reverse_proxy dashboard:3000
```

```
# }
```

```
# 3. Update workspace.branding.custom_domain = "outreach.clientagency.com"
```

Task Part 4: app/api/routes/workspaces.py add:

GET /workspace/info

Auth required

Returns: {

- id, name, plan, branding,

- stats: {total_campaigns, total_leads, active_accounts},

- created_at

```
}
```

✓ EXPECTED OUTPUT — Session complete when you see:

- PATCH /workspace/branding updates logo_url, primary_color, product_name in Supabase
- GET /workspace/branding/public returns branding without auth (used for login page)
- email_service.send_alert_email() sends or logs email with correct from-name from branding
- GET /workspace/info returns complete workspace overview

■ IF IT BREAKS — Tell Claude Code exactly this:

- Resend API 422: 'The from address must use a verified domain in Resend. Use alerts@resend.dev for testing'
- branding PATCH replaces instead of merges: 'Use jsonb_set in Supabase or: fetch current, merge in Python with {**current, **updates}, write back'
- GET /workspace/branding/public blocked by auth: 'This route must be added BEFORE auth middleware in main.py route registration order'

SESSION 15

Lead CRM + Pipeline + Status Management

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: All backend features complete. Building CRM layer for lead management.

Task Part 1: Enhanced leads API in app/api/routes/leads.py:

GET /leads (upgrade existing)

Add query params:

```
campaign_id, stage, search (searches full_name + company),
has_email (bool), icp_score_min (int),
sort_by (created_at|icp_score|connected_at), sort_dir (asc|desc)
limit=50, offset=0
```

Return: {leads: [...], total: int, page: int, has_more: bool}

GET /leads/{lead_id}

Auth required

Returns: full lead data + all messages (outbound + inbound) + campaign name

PATCH /leads/{lead_id}

Auth required

Body: {pipeline_stage, notes, icp_score} (all optional)

Update only provided fields

If pipeline_stage = "booked": fire_webhook("lead.meeting_booked", {lead_id, name, company})

POST /leads/bulk-update

Auth required

Body: {lead_ids: list[str], updates: {pipeline_stage: str}}

Bulk update up to 50 leads at once

Return {updated: int}

DELETE /leads/{lead_id}

Auth required, role=owner or admin only

Soft delete: set pipeline_stage = "not_interested", don't actually delete

Task Part 2: app/services/lead_scorer.py:

```
async def score_and_rank_leads(campaign_id: str, icp_description: str):
```

Batch score all unscored leads (icp_score = 0) for a campaign

Process in batches of 20 (to manage Claude API calls)

Call ai_service.score_lead_icp() for each

Update lead.icp_score in Supabase

Return {scored: int, avg_score: float}

```
async def auto_qualify_leads(campaign_id: str, min_score: int = 60):
```

Move leads with icp_score < min_score to stage = "not_interested"

Return {qualified: int, rejected: int}

New endpoint POST /campaigns/{id}/score-leads:

Dispatches lead scoring job for campaign

Returns {job_id, leads_to_score: int}

Task Part 3: Test the full end-to-end flow:

Write e2e_test.py in project root:

1. Register user + workspace
2. Add a LinkedIn account (fake li_at for test)
3. Create campaign with 3 test leads (insert directly to Supabase)
4. Score leads
5. Verify analytics endpoint returns correct counts
6. Mark one lead as "connected" and verify reply rate calculation

Print "E2E TEST PASSED" or show what failed.

✓ EXPECTED OUTPUT — Session complete when you see:

- GET /leads with search param filters by name/company correctly
- PATCH /leads/{id} updates pipeline_stage and fires webhook
- POST /campaigns/{id}/score-leads dispatches job, scores appear in Supabase
- POST /leads/bulk-update updates multiple leads in one call
- python e2e_test.py prints E2E TEST PASSED

■ IF IT BREAKS — Tell Claude Code exactly this:

→ Full-text search not working: 'Use ILIKE for search: .or(f"full_name.ilike.{search}%,company.ilike.{search}%")'

→ Bulk update hits Supabase limit: 'Split into chunks of 20: for chunk in [lead_ids[i:i+20] for i in range(0,len(lead_ids),20)]'

→ score_lead_icp costs too many Claude credits: 'Batch all 20 leads into ONE Claude call: list the profiles, ask to return scores as JSON array'

→ e2e_test.py fails at analytics step: 'Check that analytics queries filter by workspace_id — the test user must own the test campaign'

PHASE 4 Polish & Production — Days 46–90

Harden, monitor, and productize. By end of Phase 4 you have a production-grade self-serve platform.

SESSION 16

Production Hardening + Logging + Error Handling

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: Full featured platform. Making it production-ready.

Task Part 1: app/core/logging_config.py:

Structured JSON logging:

- Every API request: {method, path, status, duration_ms, workspace_id, user_id}
- Every job execution: {job_id, type, account_id, duration_ms, status}
- Every LinkedIn action: {account_id, action_type, profile_url, result}
- Every Claude API call: {tokens_used, latency_ms, fallback_used}

Use Python structlog or standard logging with custom JSON formatter.

Log to stdout (Docker logs capture it).

Log level from env: LOG_LEVEL=INFO

Task Part 2: Global error handling in main.py:

```
@app.exception_handler(Exception)
```

```
async def global_exception_handler(request, exc):
```

```
    Log the full traceback
```

```
    If 500 and SLACK_WEBHOOK_URL set: send Slack alert
```

```
    Return: {"error": "Internal server error", "request_id": uuid4()}
```

```
@app.exception_handler(HTTPException)
```

```
Return clean JSON: {"error": exc.detail, "status_code": exc.status_code}
```

Task Part 3: Rate limiting in app/core/rate_limiter.py:

Using Redis:

- Per-user: 100 API requests per minute
- Per-workspace: 1000 LinkedIn actions per hour
- /auth/login: 10 attempts per minute per IP (brute force protection)

Decorator: @rate_limit("per_user", limit=100, window=60)

Return 429 with Retry-After header when exceeded.

Task Part 4: Health check endpoint upgrade in main.py:

GET /health -> complete system health:

```
{
  status: "ok" | "degraded" | "down",
  api: "ok",
  database: "ok" | "error", (test Supabase with simple SELECT 1)
  redis: "ok" | "error", (PING Redis)
  worker: "ok" | "unknown", (check last job completed < 10 min ago)
  timestamp: ISO datetime,
  version: "1.0.0"
}
```

Task Part 5: Add to docker-compose.yml:

healthcheck for api service:

```
test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
interval: 30s
timeout: 10s
retries: 3
```

Add restart: unless-stopped to all services.

✓ EXPECTED OUTPUT — Session complete when you see:

- Every API request logs JSON to stdout (visible in docker compose logs api)
- Intentionally cause an error — verify Slack receives an alert
- Hit /auth/login 11 times fast — 11th returns 429 with Retry-After header
- GET /health returns {status: ok, database: ok, redis: ok}
- docker compose ps shows all services as healthy

■ IF IT BREAKS — Tell Claude Code exactly this:

→ structlog ImportError: 'Use standard logging instead: import logging; logging.basicConfig(format='%(message)s')'

→ Rate limiter blocks legitimate requests: 'Increase per-user limit to 300/minute for development, reduce to 100 before production'

→ Health check shows database: error: 'Test Supabase connection: run python -c "from app.core.database import get_client; import asyncio; asyncio.run(get_client().table(\'workspaces\').select(\'count\').execute())"

SESSION 17

Self-Serve Onboarding + Billing Hooks

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: Production-hardened. Adding self-serve signup and billing integration stubs.

Task Part 1: Self-serve signup flow:

POST /auth/register (upgrade):

After user + workspace created:

1. Send welcome email (via email_service) with onboarding checklist

2. Create default campaign template in their workspace
3. Log signup event to analytics (just a Supabase insert for now)

GET /onboarding/checklist

Auth required

Returns completion status of onboarding steps:

```
{
  steps: [
    {id: "add_account", title: "Add your LinkedIn account", completed: bool},
    {id: "create_campaign", title: "Create your first campaign", completed: bool},
    {id: "add_leads", title: "Add your first 10 leads", completed: bool},
    {id: "send_first", title: "Send your first connection request", completed: bool},
  ],
  completion_pct: int
}
```

Each step checks Supabase for existence of the resource.

Task Part 2: Billing stubs (structure for Razorpay later):

app/services/billing.py – stub implementation:

```
async def get_workspace_plan(workspace_id: str) -> str:
```

Returns workspace.plan ("starter"|"pro"|"agency")

```
async def check_feature_access(workspace_id: str, feature: str) -> bool:
```

feature options: "multi_account", "white_label", "email_enrichment", "ai_messages"

Plan gates:

starter: only ai_messages

pro: ai_messages + multi_account + email_enrichment

agency: all features

Return True/False

Decorator: @require_plan("pro") for endpoints that need paid plan:

If workspace on lower plan: return 402 with {"error": "Upgrade required", "required_plan": "pro", "upgrade_url": "/billing/upgrade"}

Apply decorator to:

POST /accounts (>1 account requires pro)

PATCH /workspace/branding (requires agency)

POST /campaigns/{id}/enrich-all (requires pro)

Task Part 3: Add to analytics:

GET /analytics/usage

Auth required

Returns current month usage vs plan limits:

```
{
  plan: str,
  limits: {campaigns: int, accounts: int, leads_per_month: int},
  used: {campaigns: int, accounts: int, leads_this_month: int},
  pct_used: {campaigns: float, accounts: float, leads: float}
}
Plan limits: starter={campaigns:3, accounts:1, leads:500},
              pro={campaigns:10, accounts:5, leads:2000},
              agency={campaigns:999, accounts:999, leads:99999}
```

✓ EXPECTED OUTPUT — Session complete when you see:

- POST /auth/register sends welcome email (or logs it to console)
- GET /onboarding/checklist returns correct completion status for new user
- POST /accounts with second account on starter plan returns 402
- GET /analytics/usage returns plan limits and current usage
- check_feature_access() returns False for starter plan trying to access white_label

■ IF IT BREAKS — Tell Claude Code exactly this:

→ 402 decorator blocks owner from their own account: 'Exempt role=owner from plan gates during beta — add: if user["role"] == "owner": return await func(*args, **kwargs)'

→ Onboarding checklist always shows 0%: 'Check queries use workspace_id from JWT — not hardcoded or from request body'

→ Welcome email not sending: 'Add LOG_EMAILS=true to .env and log the full email content to console instead of failing'

SESSION 18

Final Integration Test + Deployment Script

▼ COPY THIS ENTIRE BLOCK AND PASTE INTO CLAUDE CODE

Context: Everything built. Final integration test and deploy to production VPS.

Task Part 1: Complete integration test suite – integration_test.py:

Run this test sequence end-to-end against a running local stack:

TEST 1 - Auth flow:

Register new user, login, verify /auth/me, verify JWT has workspace_id

TEST 2 - Account management:

Add LinkedIn account (use fake li_at="fake_token_test"),
verify it appears in GET /accounts,
verify health check returns structured response (not crash)

TEST 3 - Campaign lifecycle:

Create campaign, verify status=draft


```
POST /campaigns/{id}/start, verify status=active
POST /campaigns/{id}/pause, verify status=paused
DELETE campaign that is in draft state, verify deleted
```

TEST 4 - Lead operations:

```
POST /leads/import with test CSV (generate 5 fake leads in test)
Verify GET /leads returns 5 leads
PATCH /leads/{id} with stage=connected, verify update
GET /leads?stage=connected returns only connected leads
```

TEST 5 - Analytics:

```
GET /analytics/campaign/{id} returns object with all required keys
GET /analytics/workspace returns correct active_campaigns count
GET /analytics/usage returns plan limits
```

TEST 6 - Webhooks:

```
Register webhook URL (use https://webhook.site for testing)
Update lead to connected
Verify webhook was fired (check webhook.site or just verify no error)
```

TEST 7 - Workspace:

```
PATCH /workspace/branding with {product_name: "TestAgency"}
GET /workspace/branding returns updated name
```

Print "ALL TESTS PASSED" or show failing test details.

Use httpx with a test base_url from env TEST_API_URL=http://localhost:8000

Task Part 2: deploy.sh – VPS deployment script:

```
#!/bin/bash

# Run on fresh Ubuntu 24.04 VPS (Hetzner/DigitalOcean)
set -e

echo "Installing Docker..."
curl -fsSL https://get.docker.com | sh
usermod -aG docker $USER

echo "Cloning repo..."
git clone $REPO_URL /opt/phantom-x
cd /opt/phantom-x

echo "Setting up env..."
cp .env.example .env
echo "Edit /opt/phantom-x/.env with your values, then run: docker compose up -d"
```

```

echo "Setting up auto-restart on reboot..."
systemctl enable docker

echo "Setting up log rotation..."
cat > /etc/logrotate.d/phantom-x << EOF
/var/lib/docker/containers/*/*.log {
    rotate 7
    daily
    compress
    missingok
    delaycompress
}
EOF

echo "Deployment ready. Next: fill .env, then: docker compose up -d"

```

Task Part 3: README.md — complete setup guide:

Cover: prerequisites, environment setup, first run, adding LinkedIn account, creating first campaign, connecting n8n, upgrading to production domain. Keep it under 200 lines. Developer-focused, not marketing copy.

✓ EXPECTED OUTPUT — Session complete when you see:

- python integration_test.py prints ALL TESTS PASSED
- Any failing test shows exactly which assertion failed and why
- deploy.sh runs without errors on fresh Ubuntu (test in a local VM or DO droplet)
- README.md covers all setup steps in correct order
- docker compose up -d after following README starts everything correctly

■ IF IT BREAKS — Tell Claude Code exactly this:

- Test fails at campaign delete (409 or 404): 'Add check: only allow DELETE if campaign.status in ["draft", "completed"] — return 409 otherwise with clear message'
- Webhook test fails (no external URL): 'Replace webhook.site with a local mock: run python -m http.server 9999 in another terminal, use http://host.docker.internal:9999 as webhook URL'
- deploy.sh fails on git clone: 'Make sure REPO_URL is set: export REPO_URL=https://github.com/yourusername/phantom-x.git'
- Integration test has auth issues between tests: 'Store token as module-level variable after TEST 1 and reuse in all subsequent tests'

QR Quick Reference — Commands & Troubleshooting

Most Common Commands

What	Command
Start full stack	<code>docker compose up -d</code>
View API logs live	<code>docker compose logs -f api</code>
View worker logs live	<code>docker compose logs -f playwright-worker</code>
Stop everything	<code>docker compose down</code>
Rebuild after code change	<code>docker compose up -d --build api</code>
Open Claude Code	<code>cd phantom-x && claude</code>
Run integration test	<code>python integration_test.py</code>
Check Redis queue length	<code>docker compose exec redis redis-cli LLEN phantom_x:jobs</code>
Check job counts in Supabase	Open Supabase dashboard > Table Editor > jobs > filter status
Hard reset (DANGER)	<code>docker compose down -v</code> (deletes all volumes including Redis data)

If Claude Code Gets Confused (Context Reset)

■ If Claude Code starts giving wrong code or contradicting itself, type: 'Let us reset. Read the files in the current directory first, then I will give you the next task.' This makes it re-read your actual codebase before continuing.

Session Handoff Template

At the start of each session after S01, use this preamble:

```
You are working on Phantom-X, a LinkedIn automation platform.
Project is at ~/phantom-x/. The stack is: FastAPI + Playwright + Supabase + Redis + Docker.
Sessions completed so far: [LIST COMPLETED SESSIONS e.g. S01, S02, S03]
What works: [LIST e.g. auth, accounts API, browser engine]
Now I need you to do: [PASTE SESSION PROMPT BELOW]
```

18 Sessions · 4 Phases · One product that beats PhantomBuster

Start with Session 01. Ship Phase 1 in 7 days. First client by Day 21.

